

University of Central Punjab
Faculty of Information Technology
Object Oriented Programming
Fall 2025

Lab 07

Topic

- Classes in C++, access specifiers, objects, methods, constructors, destructor, Passing objects as arguments, Returning Objects, Operator overloading.

Objective

Making students familiarize with the concepts of constructor, overloaded constructor, getter and setter functions, Passing objects as arguments, Returning Objects and Operator overloading.

CLO

Apply object-oriented programming principles to implement real-world problems.

Instructions:

- Indent your code.
- Comment your code.
- Use meaningful variable names.
- Plan your code carefully on a piece of paper before you implement it.

- Name of the program should be same as the task name. i.e. the first program should be Task_1.cpp
- **void main() is not allowed. Use int main()**
- **You are not allowed to use any built-in functions**
- **You are required to follow the naming conventions as follow:**
 - **Variables:** firstName; (no underscores allowed)
 - **Function:** getName(); (no underscores allowed)
 - **ClassName:** BankAccount (no underscores allowed)

Students are required to complete the following tasks in lab timings.

Task 1:ComplexNumber Class

Problem Statement:

Create a class ComplexNumber to represent complex numbers with the following private attributes:

- real (int)
- imag (int)

Overload the following operators:

- + to add two complex numbers
ComplexNumber operator+(const ComplexNumber& other);
- - to subtract them
ComplexNumber operator-(const ComplexNumber& other);
- * to multiply
ComplexNumber operator*(const ComplexNumber& other);
- / to divide (bonus)
ComplexNumber operator/(const ComplexNumber& other); // (optional)
- % (you may implement a custom behavior, e.g., modulus of real parts)
ComplexNumber operator%(const ComplexNumber& other); // e.g., mod real

part

Sample Output:

```
ComplexNumber c1(3, 2);  
ComplexNumber c2(1, 4);  
ComplexNumber c3 = c1 + c2;
```

Task 2: Student Class

Problem Statement:

Create a class Student with attributes name, rollNo, and GPA.

- string name;
- int rollNo;
- float gpa;

Overload the following operators:

- == and != to compare if two students are the same (based on roll number)
bool operator==(const Student& other);
bool operator!=(const Student& other);
- >= and <= to compare based on GPA
bool operator>=(const Student& other);
bool operator<=(const Student& other);

Sample Output:

```
Student s1("Ali", 101, 3.7);  
Student s2("Sara", 102, 3.9);
```

```
if(s1 <= s2) cout << "Sara has better GPA.";
```

Task 3: Counter Class

Problem Statement:

Create a class Counter that holds a number.

- value int;

Overload the following operators:

- ++ and -- (both prefix and postfix)
Counter operator++(); // prefix increment
Counter operator++(int); // postfix increment
Counter operator--(); // prefix decrement
Counter operator--(int); // postfix decrement

- Unary + to return the absolute value
Counter operator+(); // returns absolute value
- Unary - to negate the value
Counter operator-(); // returns negated value

Sample Output:

Counter c(5);

++c;

-c;

Task 4: Student Class

Use the Student class with the following private attributes.

- string name;
- int rollNo;
- float gpa;

Overload the following operators:

- >> to input data from user
friend istream& operator>>(istream& in, Student& s);
- << to display data
friend ostream& operator<<(ostream& out, const Student& s);

Sample Output:

Student s;

cin >> s;

cout << s;